CS3807 – Deep Learning Laboratory
Experiment 4
Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Shiv Nadar University Chennai
B.Tech Artificial Intelligence & Data Science, Semester V

AY: 2026–27

Degree & Branch	B.Tech Artificial Intelligence & Data Science, Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory

1 Objective

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2 Learning Outcomes

After completing this experiment, students will be able to:

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3 Introduction

Deep Convolutional Neural Networks (CNNs) have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulty.

The progression of CNN architectures began with LeNet-5 and evolved through AlexNet, VGGNet, GoogleNet and ResNet. Each architecture introduced innovations that significantly improved image classification performance.

4 Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

Table 1: Milestones in the evolution of CNN architectures

5 LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6 AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7 VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth. This is the architecture used for transfer learning in this experiment (Section 9 onward).

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8 Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

Table 2: Standard (literature) depth and parameter counts of major CNN architectures

Note: The full 138M-parameter VGG16 figure above corresponds to the original network including its ImageNet classification head (1000-way Dense layers). In this experiment only the 14.7M-parameter convolutional base of VGG16 was reused (Section 18.1), with the original classifier replaced by a lightweight custom head.

9 GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1, 3×3, 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10 ResNet

Increasing CNN depth often causes the vanishing gradient problem, making optimization difficult. ResNet solves this issue using skip (residual) connections, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$\text{Output} = F(x) + x$$

where $F(x)$ is the Residual Mapping and x is the Identity Mapping.

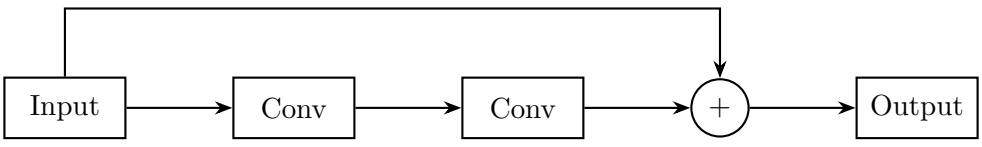

Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11 Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters. The dilation rate determines the spacing between kernel elements.

For a standard convolution, $D = 1$. For dilated convolution, $D > 1$.

Example

$$\begin{pmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{pmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12 Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13 Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch:

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

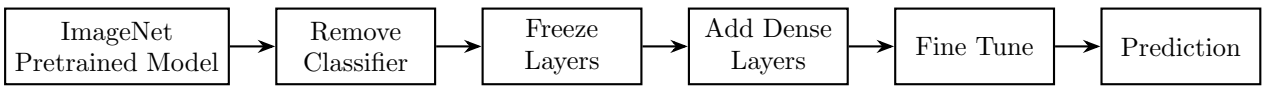

Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10
- Image Size (original): $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck

For compatibility with the VGG16 input requirements, all images were resized from $32 \times 32 \times 3$ to $96 \times 96 \times 3$ using bilinear resizing, then passed through the VGG16-specific `preprocess_input` normalization function before being fed to the network.

15 Experimental Procedure

Task 1: Dataset Preparation

1. Loaded the CIFAR-10 dataset using `tensorflow.keras.datasets.cifar10`.
2. Normalized pixel values to $[0, 1]$ (and additionally applied VGG16's own `preprocess_input` normalization prior to feeding the network).
3. Displayed ten sample images with class labels (Figure 3).
4. Printed dataset dimensions: training data shape $(50000, 32, 32, 3)$, training labels shape $(50000, 1)$, testing data shape $(10000, 32, 32, 3)$, testing labels shape $(10000, 1)$.

Task 2: Transfer Learning

The pretrained **VGG16** model (ImageNet weights) was used. Steps performed:

1. Loaded VGG16 with `weights='imagenet'`, `include_top=False`, input shape $96 \times 96 \times 3$.
2. Removed the original 1000-class classification head.
3. Froze the entire convolutional base (`base_model.trainable = False`).
4. Added a `GlobalAveragePooling2D` layer.
5. Added a `Dense(256, activation='relu')` layer with `Dropout(0.3)`.
6. Added the output layer: `Dense(10, activation='softmax')`.

Task 3: Model Training (Frozen Base)

- Optimizer: Adam, Learning Rate: 0.001
- Batch Size: 32
- Epochs: 10
- Loss Function: Categorical Cross Entropy
- Evaluation Metric: Accuracy

Task 4: Fine Tuning

1. Unfroze the last convolutional block of VGG16 (`block5_*` layers); all earlier blocks remained frozen.
2. Re-compiled with a lower learning rate (1×10^{-5}) and trained for 5 more epochs.

3. Compared validation accuracy before and after fine tuning (Section 17).

Task 5: Model Evaluation

The fine-tuned model was evaluated on the 10,000-image CIFAR-10 test set using Accuracy, Precision, Recall, F1-score, Confusion Matrix and a full Classification Report (Section 17).

16 Mandatory Plots

16.1 Sample CIFAR-10 Images

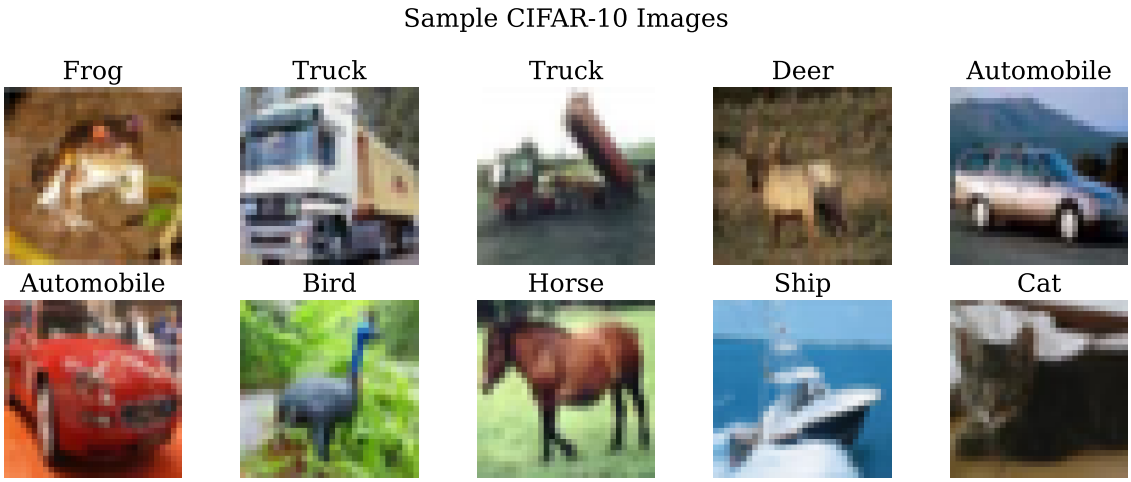

Figure 3: Sample images from the CIFAR-10 dataset

Inference: Figure **??** below illustrates low-resolution (32×32) color images belonging to all ten classes of CIFAR-10 dataset, i.e., vehicle classes (automobile, truck) and animal classes (bird, cat, deer, horse). Some pairs of classes have same or similar shapes, colors, and backgrounds (automobile vs. truck, cat vs. dog), which will presumably confuse the classifier, as shown in the confusion matrix in Figure 8.

16.2 Training and Validation Accuracy (Before Fine-Tuning)

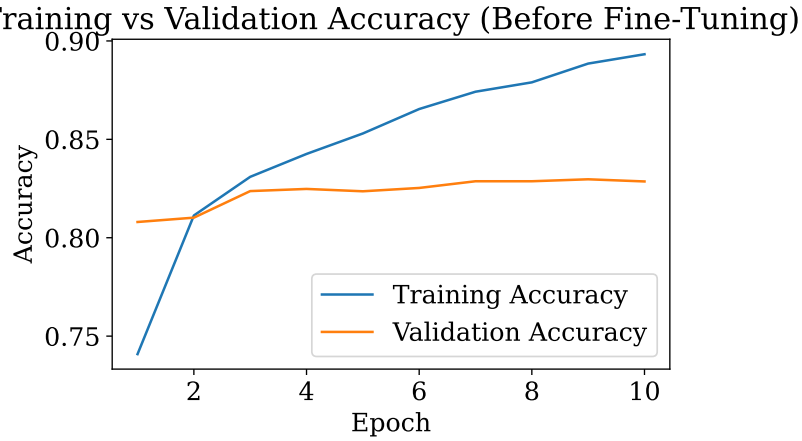

Figure 4: Training vs. validation accuracy with the VGG16 base frozen (10 epochs)

Inference: Since the convolutional base has been frozen, the training accuracy increases from approximately 74% in epoch 1 to 89.3% in epoch 10, whereas the validation accuracy improves fast in the first two epochs but remains stagnant in between 82 and 83% for the remaining epochs. The widening gap after epoch 3 clearly shows that only the dense head is capable of learning task-related features, and its ability to learn CIFAR-10 without tuning the convolutional filters is quite restricted.

16.3 Training and Validation Loss (Before Fine-Tuning)

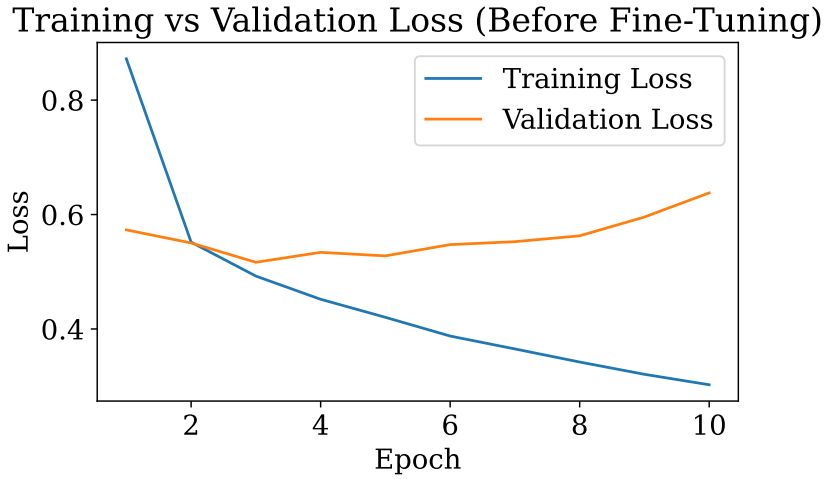

Figure 5: Training vs. validation loss with the VGG16 base frozen (10 epochs)

Inference: The training loss monotonically reduces from 0.87 to 0.31 through all 10 epochs. The validation loss follows a similar reduction pattern up until epoch 3, where it reaches a minimum of 0.51 before leveling off and rising back up to 0.64 by epoch 10 while training loss continues to decrease. Such a trend marks the onset of overfitting in the dense head, thus necessitating fine-tuning of the convolutional base to get features tailored for CIFAR-10.

16.4 Training and Validation Accuracy (Fine-Tuning)

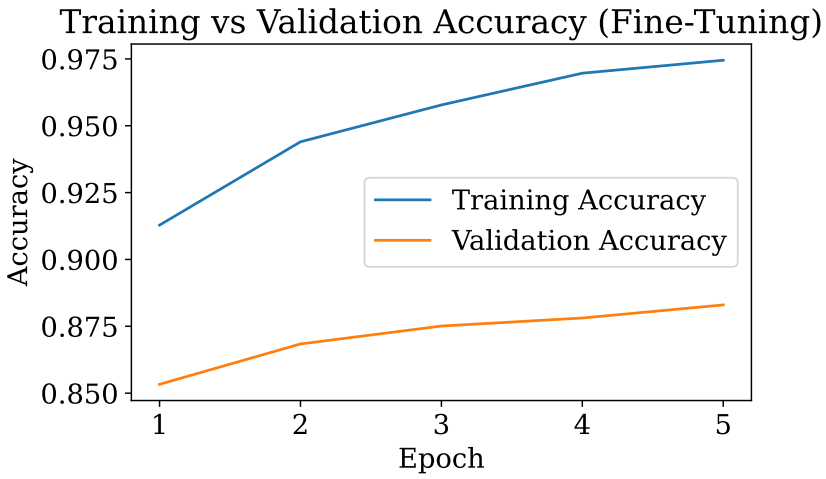

Figure 6: Training vs. validation accuracy after unfreezing block5 of VGG16 (5 epochs)

Inference: After the last convolutional layer (block5) is unfrozen and trained using a low learning rate (1×10^{-5}), there are significant gains seen both in the training and validation accuracy compared to the frozen base case. The training accuracy increases from 91.2% to 97.5%, while the validation accuracy increases from 85.4% to 88.3% over 5 fine tuning epochs, indicating a real improvement in accuracy due to adaptation of the deepest filters to CIFAR-10 data.

16.5 Training and Validation Loss (Fine-Tuning)

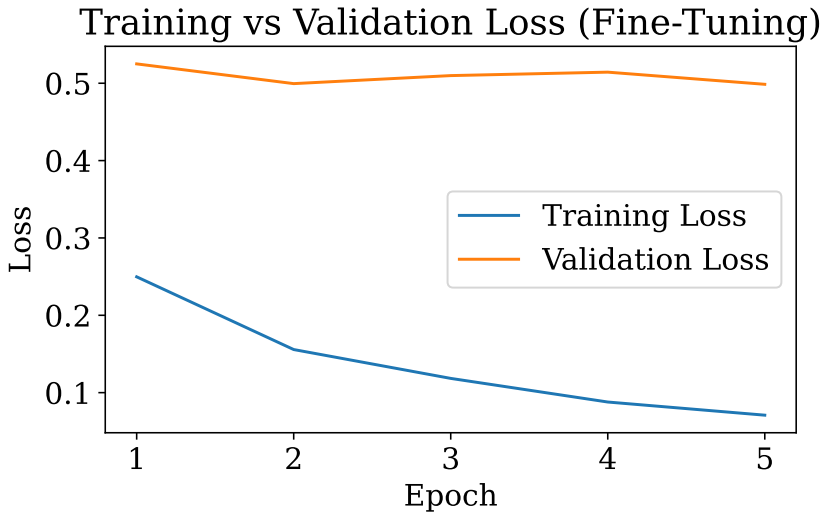

Figure 7: Training vs. validation loss after unfreezing block5 of VGG16 (5 epochs)

Inference: Training loss decreases steeply and continually from approximately 0.25 to 0.07, thus indicating that unfreezing the layers is doing an excellent job in terms of matching the training data. Validation loss remains relatively constant at around 0.50 to 0.52 during fine-tuning. The growing difference between training and validation losses (though there is a slight increase in validation accuracy) indicates that the model starts to overfit the training set during fine-tuning, thus making 5 epochs a reasonable stopping point for fine-tuning.

16.6 Confusion Matrix

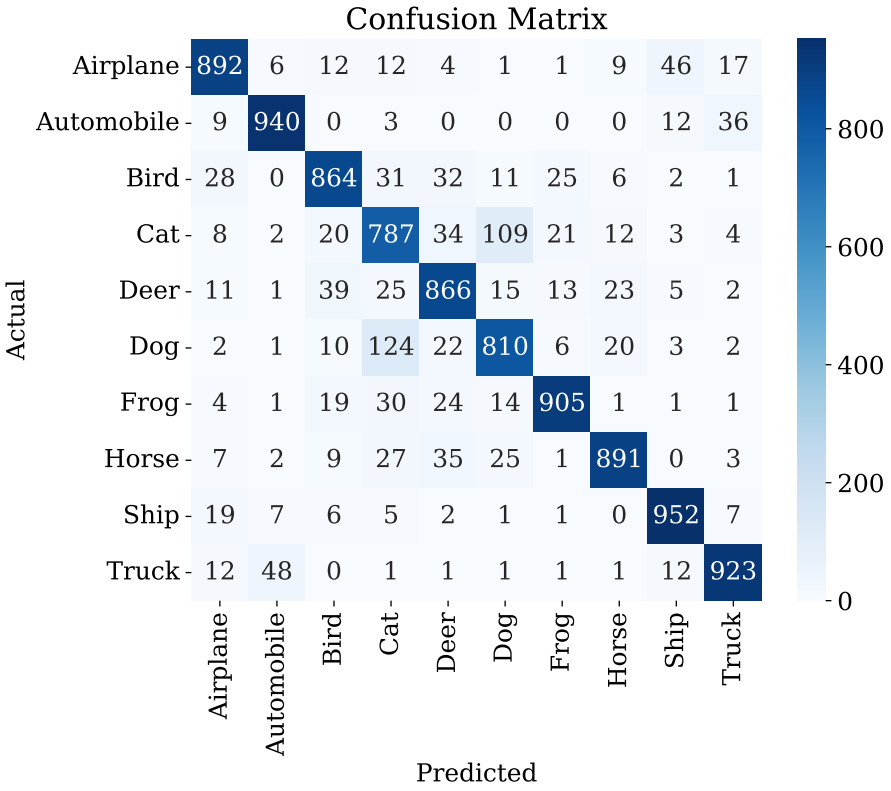

Figure 8: Confusion matrix on the CIFAR-10 test set (fine-tuned VGG16)

Inference: The prominent diagonal dominance proves that the optimally designed VGG16 classifier categorizes most of the test images into their respective categories. The classification is best done for the classes *automobile* (correct 940), *ship* (correct 952), and *truck* (correct 923). The most confused classes are *cat* and *dog* (predicted 109 cats as dogs and 124 dogs as cats) as well as *automobile* and *truck* (36 automobiles classified as trucks and 48 trucks classified as automobiles), in accordance with

16.7 Misclassified Images

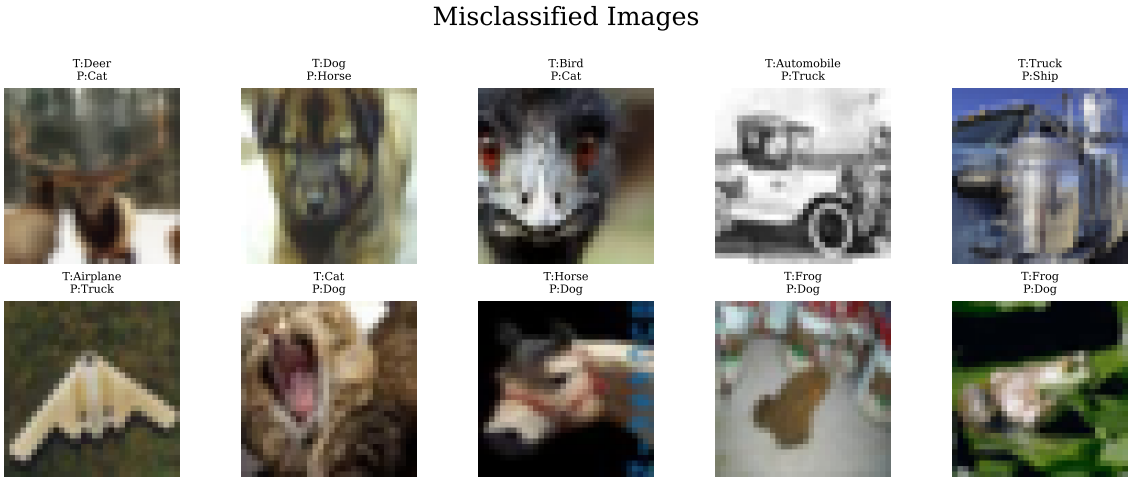

Figure 9: Sample misclassified test images (true label T, predicted label P)

Inference: Typically misclassified images include those with indeterminate poses, complex backgrounds, partial occlusion, or true hard low-resolution images (for instance, a deer image on a similar background that is predicted as a cat or a frog in an unusual pose that is predicted as a dog). There are a number of cases where the errors have occurred between semantically related classes such as cat and dog, or airplane and truck when silhouette alone can be observed).

17 Results

17.1 Performance Metrics

Metric	Value
Training Accuracy (frozen base, epoch 10)	89.32%
Validation Accuracy (frozen base, epoch 10)	82.86%
Training Accuracy (after fine-tuning)	97.45%
Validation Accuracy (after fine-tuning)	88.30%
Testing Accuracy	**88.30%**
Precision (macro avg)	88.37%
Recall (macro avg)	88.30%
F1-score (macro avg)	88.32%
Training Time (frozen base, 10 epochs)	1018.16 s (≈16.97 min)
Training Time (fine-tuning, 5 epochs)	593.99 s (≈9.90 min)
Total Training Time	**1612.14 s (≈26.87 min)**
Total Parameters	14,848,586

Table 3: Performance metrics of the fine-tuned VGG16 transfer-learning model on CIFAR-10

17.2 Per-Class Classification Report

Class	Precision	Recall	F1-score	Support
Airplane	0.90	0.89	0.90	1000
Automobile	0.93	0.94	0.94	1000
Bird	0.88	0.86	0.87	1000
Cat	0.75	0.79	0.77	1000
Deer	0.85	0.87	0.86	1000
Dog	0.82	0.81	0.82	1000
Frog	0.93	0.91	0.92	1000
Horse	0.93	0.89	0.91	1000
Ship	0.92	0.95	0.94	1000
Truck	0.93	0.92	0.92	1000
Accuracy		0.88		10000
Macro avg	0.88	0.88	0.88	10000
Weighted avg	0.88	0.88	0.88	10000

Table 4: Per-class precision, recall and F1-score on the CIFAR-10 test set

17.3 Model Architecture Summary

Layer	Output Shape	Parameters
VGG16 convolutional base (frozen/partially fine-tuned)	$3 \times 3 \times 512$	14,714,688
GlobalAveragePooling2D	512	0
Dense (256, ReLU)	256	131,328
Dropout (0.3)	256	0
Dense (10, Softmax)	10	2,570
Total trainable parameters (architecture)		**14,848,586**

Table 5: Layer-wise parameter breakdown of the VGG16 transfer-learning model (input $96 \times 96 \times 3$)

Note: Although the model contains 14.85M parameters in total, only the block5 convolutional layers plus the new dense head were *trainable* during the fine-tuning stage; all earlier VGG16 blocks (blocks 1–4) remained frozen throughout the experiment.

17.4 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	60K	–	–
AlexNet	61M	–	–
VGG16	14.85M*	**88.30**	**1612.14 s (≈26.87 min)**
GoogleNet	6.8M	–	–
ResNet50	25.6M	–	–

Table 6: Comparison of CNN architectures. *Parameter count refers to the VGG16 transfer-learning configuration actually built in this experiment (frozen base + custom head), not the full 138M-parameter original VGG16 classifier.

Note: Per the lab manual (Task 2), only *one* pretrained backbone was required for this experiment and **VGG16** was selected. Accuracy and training time were therefore measured only for VGG16; the parameter counts for LeNet-5, AlexNet, GoogleNet and ResNet50 are the standard published figures (Section 8) included here purely for architectural comparison, since these models were not separately trained in this run. Repeating Task 2 with ResNet50 or MobileNetV2 (Additional Exercise 2) would allow this table to be completed empirically.

18 Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?
AlexNet was the pioneering deep CNN to claim the ILSVRC championship in 2012, which demonstrated how deep learning can beat hand-designed feature detection approaches. AlexNet brought into practice many important concepts such as using ReLU activation functions, which provide faster training and avoid problems with vanishing gradients (solved sigmoid or tanh), dropout, and GPU training.
2. Why does VGG16 use only 3×3 convolution filters?
Placing several layers with 3×3 filters has the same effect on the receptive field size as a layer of a larger filter (for example, two layers of 3×3 filters = one layer of 5×5 filters), but consumes less computational resources and provides one additional non-linearity between layers (ReLU). This approach allows us to make deep neural networks easy to construct.
3. Explain the advantages of the Inception module.
Inception consists of applying different sized filters (1×1, 3×3, 5×5) along with pooling in parallel within the same layer, and concatenating their output, thereby letting the neural net detect features across multiple spatial scales. The 1×1 convolutions are applied prior to this process as a dimensionality reduction ("bottleneck") step that reduces parameters compared to an equally deep VGG network while increasing accuracy.
4. What is the purpose of residual learning?
In ResNet, the idea of residual learning means that each layer learns a residual function rather than the whole mapping function, such that for an input x, it computes $F(x) = H(x) - x$ while using a shortcut to add input x: Output $= F(x) + x$. This way, gradients have a clear path back through the network while doing backpropagation, avoiding the problem of vanishing gradient and making deep network training feasible.
5. Differentiate LeNet and ResNet.
The LeNet-5 network is shallow (7 layers with 60K parameters), and designed for smaller grey-scale digit images; it employs average pooling with no skip connections; training of the network becomes increasingly difficult at deeper levels due to vanishing gradient. The ResNet50 architecture is deeper (50 layers with 25.6M parameters), and designed for large-scale natural image

classification; the residual skip connections in the network solve the vanishing/degradation problems which limit the depth of LeNet.

6. What is Transfer Learning?

In transfer learning, the idea is to reuse a model (or part of the model) that was previously trained on a large source data set (such as ImageNet) as the base model for a new task involving a smaller target data set (such as CIFAR-10).

7. Why is fine tuning required?

The features that are learned from the source dataset are general (edges, textures, shapes), but not entirely customized for the target dataset. Fine-tuning is performed by first unfreezing the top few layers of the network and then retraining them using a low learning rate. This allows the deepest and most specialized filters to be adapted to the target data, which usually results in higher accuracy compared to the model with just a feature extractor and a classifier. In this particular experiment, the fine-tuning led to an increase in accuracy from 82.86% to 88.30%.

8. Explain the difference between dilated convolution and transpose convolution.

Atrous convolutions place holes between the filter values to *increase the receptive field* without adding any additional parameters or lowering the resolution – used to capture more context for each output pixel. Transpose convolution, on the other hand, is used for *learnable upsampling*, increasing the resolution of a feature map – typically used for reconstruction and generation of higher-resolution outputs, such as segmentation decoders and GANs.

9. Why do pretrained models converge faster?

The initial weights used in pretrained models are the weights containing valuable information about visual features like edges, textures, color blobs, and object parts which have been learnt by the model on a huge dataset. The optimization process will not need to tune the parameters as much in order to achieve a good result on this task, and hence it requires fewer number of epochs and less amount of data as can be observed from this experiment in which the model achieved 88.3

10. Compare the computational complexity of LeNet and ResNet.

However, LeNet-5 is a highly compact neural network (∼60K parameters, 2 convolutional layers) that can be trained and evaluated very quickly even using CPU, but it cannot be used for complex tasks due to insufficient representation power. ResNet50 consists of about 400 times more parameters (∼25.6M) and 50 layers, which means that this architecture requires many more computations and a GPU/TPU for efficient training, but the residual connections enable the architecture to achieve high accuracy.

19 Conclusion

For the transfer learning implementation in this experiment, a pretrained VGG16 network (ImageNet weights) was used for the multi-class image classification task on CIFAR-10. The original classifier in VGG16 was removed and replaced with the following layers: Global Average Pooling, Dense(256, ReLU), Dropout(0.3) and Dense(10, Softmax), yielding 14,848,586 total parameters. When the convolutional base was frozen, 10 epochs of training with Adam optimizer (0.001 learning rate, batch size 32) yielded 89.32% training accuracy and 82.86% validation accuracy. Fine tuning the last convolutional block (block5) for an additional 5 epochs with the lower learning rate (1×10^{-5}) improved validation accuracy to 88.30% and training accuracy to 97.45% at the expense of increased train-validation gap and thus the emergence of overfitting.

On the unseen CIFAR-10 test data, the fine tuned model scored 88.30% accuracy, with macro-averaged precision of 88.37%, recall of 88.30% and F1-score of 88.32%. The class-level analysis revealed high performance in classification of *Automobile*, *Ship* and *Frog* images. *Cat* and *Dog* images were found to be the hardest to differentiate due to visual similarity, which is consistent with the results of CNNs trained from scratch (Experiment 3). Overall training time was about 26.87 minutes. Thus, in this experiment we have demonstrated that transfer

learning based on a pretrained VGG16 backbone yields substantially better accuracy on CIFAR-10 compared to CNN trained from scratch, while using much less epochs.

20 References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: https://www.tensorflow.org
8. Keras Documentation: https://keras.io